



Licenciatura em Engenharia Informática e Multimédia
Infraestruturas Computacionais Distribuídas

Grupo:

Gianni Floriddia – nº51945
Pedro Marques – nº51959
Rodrigo Neves – nº51452

Data: 27/04/2025

Índice:

1 - Diagrama Geral da Arquitetura	2
2.1 - Protocolo utilizado:	4
2.2 - Porto utilizado:	4
2.3 - Ligação:	4
2.4 - Concorrência:	4
3 - Componentes e Funcionalidades	5
3.1 Servidor TCP [Concorrente]	5
3.2 Jogador Conectado [Concorrente]	5
3.3 Gerir Jogadores [Iterativo]	5
3.4 JogadoresDataBase [Iterativo]	5
3.5 SalaDeEspera [Concorrente]	6
3.6 Jogador Perfil [Iterativo]	6
3.7 ConexaoJogador [Concorrente]	6
3.8 SalaDeJogo [Concorrente]	6
3.9 Jogo e Jogador [Iterativo e Concorrente]	6
3.10 Util [Iterativo]	7
4 - Estruturas de Dados Persistentes	8
5 - Protocolo de Aplicação	9
5.1 Formato das mensagens:	9
5.2 Propósito das mensagens:	9
5.3 Temporização	10
6 - Exemplos da Aplicação	11
7 - Conclusões	12

1 - Diagrama Geral da Arquitetura

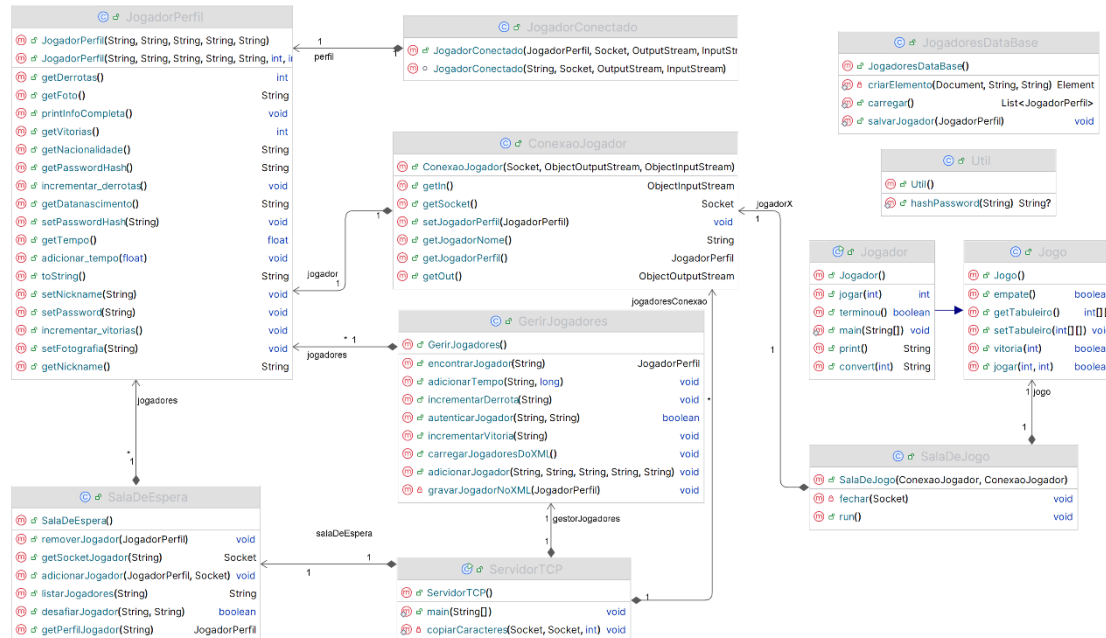


Figura 1: UML

O sistema apresenta o jogo de 15 em linha, onde os jogadores podem autenticar-se, jogar contra outros em tempo real e acompanhar o seu desempenho.

A classe ServidorTCP gerência conexões de clientes e cria sessões para cada jogador. Este servidor utiliza o método copiarCaracteres() para transferir dados entre sockets. A gestão dos jogadores é realizada pela classe GerirJogadores, responsável pelo registro, autenticação e atualização de estatísticas como vitórias e derrotas. As informações dos jogadores são persistidas em formato XML através do método gravarJogadorNoXML().

A classe JogadoresDataBase manipula a leitura e escrita desses dados, criando elementos XML e convertendo-os em objetos da classe JogadorPerfil, que armazena informações como nacionalidade, data de nascimento e estatísticas de jogo. Cada JogadorPerfil possui métodos para gerenciar seus dados, como getFoto(), setNickname() e incrementarVitorias().

A conexão de um jogador é gerenciada pela classe JogadorConectado, que associa um perfil a um socket e streams de

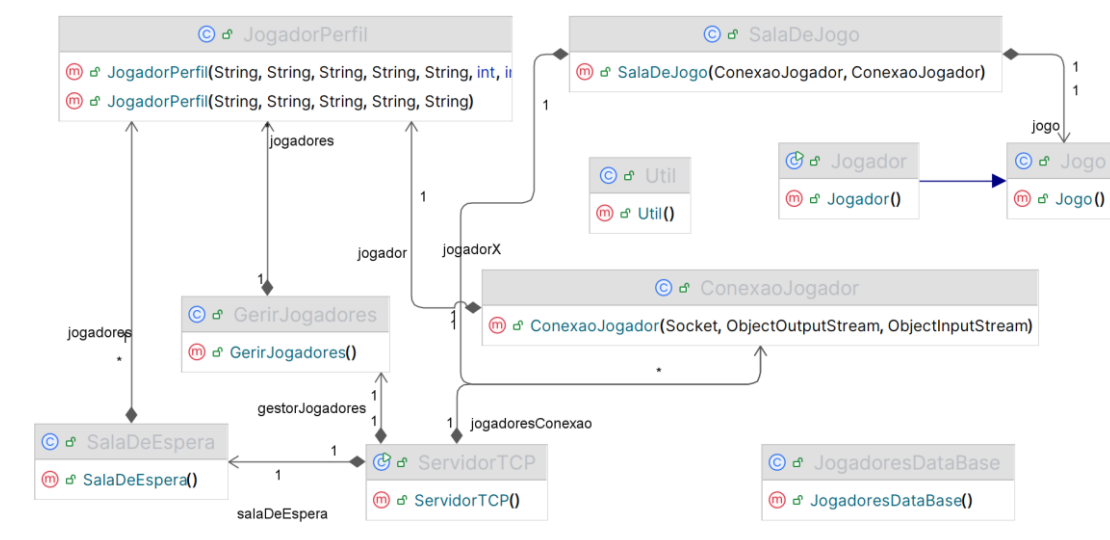
entrada/saída. A classe `ConexaoJogador` facilita a comunicação entre o servidor e o cliente, oferecendo métodos como `getJogadorNome()` e `getJogadorPerfil()`.

Os jogadores são organizados na `SalaDeEspera`, onde aguardam por adversários. Esta sala permite adicionar e remover jogadores, além de listar os participantes disponíveis. Quando dois jogadores estão prontos, a `SalaDeJogo` gerencia a partida, controlando o fluxo do jogo através das classes `Jogo` e `Jogador`.

O jogo propriamente dito é implementado nas classes `Jogo` e `Jogador`, que incluem métodos para realizar jogadas (`jogar()`), verificar vitórias (`vitória()`) e empates (`empate()`), além de gerenciar o tabuleiro de jogo (`getTabuleiro()`, `setTabuleiro()`).

A classe `Util` fornece funcionalidades auxiliares, incluindo o método `hashPassword` para criptografar senhas dos usuários, garantindo maior segurança no armazenamento de credenciais.

UML Simplificado



2 - Protocolos de Transporte

2.1 - Protocolo utilizado:

TCP (Transmission Control Protocol)

Optámos por usar TCP pois este garante confiabilidade na entrega de dados, ordem e deteta perdas ou erros, estas características não presentes no UDP possuem grande importância na troca de comandos e jogadas entre os jogadores e o servidor e entre jogadores.

2.2 - Porto utilizado:

Foi definido um porto padrão 5025 (DEFAULT_PORT), é através deste porto que todos os clientes estabelecem ligação com o servidor.

Este porto está dentro do intervalo de portos não reservados o que permite evitar conflitos com serviços conhecidos. Ex: HTTP(80).

2.3 - Ligação:

Cada jogador estabelece ligação ao servidor no momento do login. Esta ligação é mantida por todas as fases do programa. (Autenticação, Seleção de adversários, Emparelhamento e Jogo)

2.4 - Concorrência:

Para que várias ligações pudessem ser realizadas em simultâneo utilizamos uma thread por cliente denominada de lidarCliente() que recebe como argumento o socket do mesmo.



3 - Componentes e Funcionalidades

3.1 Servidor TCP [Concorrente]

- Responsável pela escuta de ligações dos clientes através do método `main()`.
- Inicia o servidor e aguarda conexões.
- Utiliza a Classe `SalaDeJogo` para criar pares de jogo e organizar o jogo durante a sua execução.
- Garante comunicação contínua com múltiplos jogadores simultaneamente.

3.2 Jogador Conectado [Concorrente]

- Representa um jogador ligado ao servidor, mantendo o estado da conexão.
- Associa um `JogadorPerfil` ao socket através do método `setJogadorPerfil()`.
- Armazena os fluxos de comunicação (`OutputStream` e `InputStream`) para que estes não tenham de ser instanciados novamente originando erros.
- Permite comunicação independente entre cada jogador e o servidor.

3.3 Gerir Jogadores [Iterativo]

- Controla o registo, autenticação e estatísticas dos jogadores.
- Utiliza métodos como `encontrarJogador()` e `autenticarJogador()`.
- Adiciona novos jogadores com `adicionarJogador()`.
- Atualiza estatísticas com `incrementarVitoria()` e `incrementarDerrota()`.
- Persiste dados com `gravarJogadorNoXML()` e `carregarJogadoresDoXML()`.
- Controla o tempo de jogo com `adicionarTempo()`.

3.4 JogadoresDataBase [Iterativo]

- Interage com o documento XML para persistência de dados.
- Cria elementos XML com `criarElemento()`.
- Gerência jogadores com `carregar()` e `salvarJogador()`.
- Converte entre formato XML e objetos `JogadorPerfil`.

3.5 SalaDeEspera [Concorrente]

- Controla os jogadores disponíveis para emparelhamento.
- Adiciona/remove jogadores com adicionarJogador() e removerJogador().
- Lista jogadores disponíveis com listarJogadores().
- Obtém sockets específicos com getSocketJogador().
- Verifica o perfil de jogadores com getPerfilJogador().
- Ajuda no gerenciamento do emparelhamento de jogadores para partidas.

3.6 Jogador Perfil [Iterativo]

- Modelo de dados com informações de cada jogador.
- Armazena nickname, dados pessoais, nacionalidade e estatísticas.
- Fornece métodos como getNickname(), getFoto() e toString().
- Implementa contadores de vitórias e derrotas com getVitorias() e getDerrotas().
- Gerencia dados de perfil como setFotografia() e setPasswordHash().

3.7 ConexaoJogador [Concorrente]

- Facilita a comunicação entre servidor e cliente.
- Gerência sockets através de getSocket() e getIn()/getOut().
- Obtém informações do jogador com getJogadorNome() e getJogadorPerfil().
- Coordena a troca de dados através de ObjectInputStream e ObjectOutputStream.

3.8 SalaDeJogo [Concorrente]

- Gerência as partidas ativas entre pares de jogadores.
- Controla o fluxo do jogo através dos métodos run() e fechar().
- Permite comunicação entre os dois jogadores durante a partida.

3.9 Jogo e Jogador [Iterativo e Concorrente]

- Implementam a lógica do jogo com métodos como jogar(), empate() e vitoria().
- Gerenciam o tabuleiro com getTabuleiro() e setTabuleiro().
- Processam as jogadas e validam resultados.

- Jogador implementa métodos como terminou() e convert() para interface com o usuário.

3.10 Util [Iterativo]

- Fornece funções auxiliares ao sistema.
- Implementa hashPassword() para segurança das senhas dos usuários.
- Evita redundância de código em operações comuns.

4 - Estruturas de Dados Persistentes

A persistência dos dados dos jogadores no servidor é assegurada através de um ficheiro XML chamado Jogadores.xml, que contém todos os perfis de utilizador registados. Este ficheiro armazena informações essenciais como o nickname, password (já encriptada), nacionalidade, fotografia (ou caminho para a imagem), número de vitórias e derrotas, tempo total de jogo e data de nascimento.

A estrutura do ficheiro XML está validada por um esquema XSD, que define o formato e os tipos de dados permitidos. O elemento raiz é <jogadores>, que pode conter um número ilimitado de elementos <jogador>. Cada jogador tem um conjunto bem definido de campos, como nickname (string), password (string encriptada), nacionalidade, fotografia, vitórias e derrotas (ambos int), tempoJogo (float) e dataNascimento (string). Esta validação garante a integridade e consistência dos dados.

A manipulação dos dados persistentes é feita através da classe JogadoresDataBase, que oferece métodos para carregar os jogadores a partir do XML (carregar()), guardar alterações (salvarJogador()), e criar novos elementos a serem adicionados ao documento (criarElemento()). Estas operações baseiam-se na API de DOM do Java para leitura e escrita de ficheiros XML.

Além disso, os objetos JogadorPerfil representam internamente cada entrada de jogador lida do XML. Sempre que um jogador inicia sessão, o seu perfil é carregado e mantido em memória, podendo ser atualizado dinamicamente (como a contagem de vitórias/derrotas ou tempo de jogo), e posteriormente persistido novamente no XML.

Esta combinação entre a definição em XSD, o armazenamento em XML e a estruturação em objetos Java permite uma gestão organizada e fiável dos dados no servidor.

```
<jogador>
  <nickname>joao298</nickname>
  <password>e8b0669e1c483d85e0a137a673db1212c210dbf64acd6d55d1521be55f5540bb</password>
  <nacionalidade>Espanhola</nacionalidade>
  <fotografia/>
  <vitórias>6</vitórias>
  <derrotas>9</derrotas>
  <tempoJogo>1200.0</tempoJogo>
  <dataNascimento>25/12/2000</dataNascimento>
</jogador>
```

5 - Protocolo de Aplicação

A comunicação entre Cliente e Servidor usa objetos Java serializados (ObjectOutputStream e ObjectInputStream).

5.1 Formato das mensagens:

- String → Para troca de mensagens de controle e comunicação (login, comandos, confirmações, mensagens do jogo).
- Integer → Para enviar o símbolo do jogador (226 para 'X' e 227 para 'O').
- Jogo → Objeto representando o estado completo do tabuleiro durante a partida.

5.2 Propósito das mensagens:

- Login:
Cliente envia um String com "nickname|password".
Servidor responde com um String de sucesso ou falha.
- Comandos após login:
 - /listar → Cliente solicita lista de jogadores disponíveis.
 - /desafiar nickname → Cliente desafia outro jogador.
O servidor responde com listas, confirmações de emparelhamento ou mensagens de espera.
- Início de Jogo:
 - Após ambos os jogadores aceitarem o desafio, o servidor cria uma Sala de Jogo.
 - Servidor envia:
 - Uma String de boas-vindas ("A partida começou! Você é o X/O.")
 - Um Integer com o símbolo atribuído.

- Durante o Jogo:
 - Cliente envia um objeto Jogo atualizado para o servidor após jogar.
 - Servidor repassa o Jogo para o adversário.
 - O fluxo alterna entre os dois jogadores até a vitória ou empate.

5.3 Temporização

- Antes do jogo:
Comunicação é síncrona — o cliente espera respostas antes de enviar novos comandos.
- Durante o jogo:
Comunicação é assíncrona e alternada: apenas quem tem o turno envia uma jogada, enquanto o outro aguarda.
- A gestão dos turnos (turnoJogadorX) é feita no servidor (SalaDeJogo), garantindo a correta alternância entre jogadores.

6 – Exemplos da Aplicação

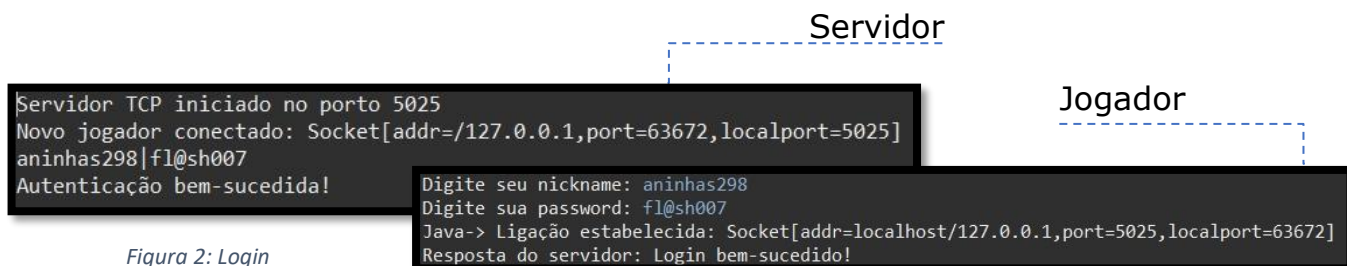


Figura 2: Login

```

/desafiar joao298
Emparelhamento confirmado com joao298! Sala de jogo criada.
Iniciando jogo com joao298
A partida começou! Você é o X.
Símbolo atribuído: X
1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15
16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30
31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45
46 | 47 | 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60
61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 | 72 | 73 | 74 | 75
76 | 77 | 78 | 79 | 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90
91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 100 | 101 | 102 | 103 | 104 | 105
106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 | 120
121 | 122 | 123 | 124 | 125 | 126 | 127 | 128 | 129 | 130 | 131 | 132 | 133 | 134 | 135
136 | 137 | 138 | 139 | 140 | 141 | 142 | 143 | 144 | 145 | 146 | 147 | 148 | 149 | 150
151 | 152 | 153 | 154 | 155 | 156 | 157 | 158 | 159 | 160 | 161 | 162 | 163 | 164 | 165
166 | 167 | 168 | 169 | 170 | 171 | 172 | 173 | 174 | 175 | 176 | 177 | 178 | 179 | 180
181 | 182 | 183 | 184 | 185 | 186 | 187 | 188 | 189 | 190 | 191 | 192 | 193 | 194 | 195
196 | 197 | 198 | 199 | 200 | 201 | 202 | 203 | 204 | 205 | 206 | 207 | 208 | 209 | 210
211 | 212 | 213 | 214 | 215 | 216 | 217 | 218 | 219 | 220 | 221 | 222 | 223 | 224 | 225

Joga X:
66
Jogada concretizada!
  
```

```

/desafiar aninhas298
Desafio enviado para aninhas298. Aguardando resposta...
Emparelhamento confirmado com aninhas298! Sala de jogo criada.
Iniciando jogo com aninhas298
A partida começou! Você é o O.
Símbolo atribuído: O
1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15
16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30
31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45
46 | 47 | 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60
61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 | 72 | 73 | 74 | 75
76 | 77 | 78 | 79 | 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90
91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 100 | 101 | 102 | 103 | 104 | 105
106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 | 120
121 | 122 | 123 | 124 | 125 | 126 | 127 | 128 | 129 | 130 | 131 | 132 | 133 | 134 | 135
136 | 137 | 138 | 139 | 140 | 141 | 142 | 143 | 144 | 145 | 146 | 147 | 148 | 149 | 150
151 | 152 | 153 | 154 | 155 | 156 | 157 | 158 | 159 | 160 | 161 | 162 | 163 | 164 | 165
166 | 167 | 168 | 169 | 170 | 171 | 172 | 173 | 174 | 175 | 176 | 177 | 178 | 179 | 180
181 | 182 | 183 | 184 | 185 | 186 | 187 | 188 | 189 | 190 | 191 | 192 | 193 | 194 | 195
196 | 197 | 198 | 199 | 200 | 201 | 202 | 203 | 204 | 205 | 206 | 207 | 208 | 209 | 210
211 | 212 | 213 | 214 | 215 | 216 | 217 | 218 | 219 | 220 | 221 | 222 | 223 | 224 | 225

Joga O:
  
```

Figura 3/4: exemplo de jogada

```

Digite seu nickname: ZêMambinho
Digite sua password: Mambo123
Java-> Ligação estabelecida: Socket[addr=localhost/127.0.0.1,port=5025,localport=63699]
Resposta do servidor: Login bem-sucedido!
Selecione um comando: /listar ou /desafiar + jogador
primeiro recebido server - jogador
/listar
joao298
aninhas298
  
```

Figura 5: exemplo de listar jogadores

7 – Conclusões

A arquitetura desenvolvida para o jogo “15 em linha” revela-se eficaz e bem-adaptada ao contexto de aplicações interativas em rede.

A organização modular do código e a estrutura de dados é baseada em XML permitem a adição de novas funcionalidades, como modos de jogo ou sistemas de pontuação mais complexos, sem a necessidade de reestruturar o projeto base.

A segurança é assegurada, em parte, através da encriptação das palavras-passe com o algoritmo SHA-256, protegendo as credenciais armazenadas, embora a ausência de mecanismos mais avançados como encriptação das comunicações ou autenticação por sessão limite a sua robustez em ambientes com maior exposição.

No que toca à transparência, o sistema esconde eficazmente a complexidade da gestão de sessões e persistência de dados do ponto de vista do utilizador, oferecendo uma interação simples e fluida.

No entanto, a tolerância a falhas pode ser considerada uma das limitações da solução, pois a dependência de um único ficheiro XML para a persistência dos dados e a ausência de mecanismos automáticos de recuperação ou backups representam potenciais pontos de falha. Ainda assim, para o âmbito em que se insere, esta arquitetura constitui uma base sólida, clara e funcional, capaz de oferecer um bom desempenho e de evoluir conforme as necessidades futuras do sistema.

Acreditamos que foi feito um bom trabalho na superação de adversidades encontradas ao longo do desenvolvimento, e concordamos que o desenvolvimento deste projeto teve uma grande contribuição para o nosso conhecimento sobre os temas abordados.